

Employee Performance Tracking System - Project Documentation

By Keerthi J K · Python · SQLite · MongoDB Atlas · Pytest · GitHub

GitHub repository: https://github.com/keerthijk193/Employee_Performance_Tracker

1. Project Overview

The Employee Performance Tracking System is a command-line application written in Python. It helps HR and team leads to:

1. Onboard new employees with their basic details.
2. Assign employees to projects and record their roles.
3. Record performance reviews for employees.
4. Generate reports on employee-project assignments and employee performance.

The main purpose of the project is to learn **polyglot persistence**: using the right kind of database for each kind of data. Structured, related data (employees, projects, assignments) is stored in a **SQL database (SQLite)**. Flexible, evolving data (performance reviews) is stored in a **NoSQL database (MongoDB Atlas)**. Python connects the two and presents everything through a simple menu.

2. Why Two Different Databases?

Data	Database	Reason
Employees, Projects, Assignments	SQLite (SQL)	Fixed columns, clear relationships (one employee, many projects), needs integrity rules such as unique email and foreign keys
Performance reviews	MongoDB (NoSQL)	Review fields can differ by role and change over time. Documents can hold lists (strengths) and new fields without changing a schema

A real example from the project: one review document has an extra field

`leadership_score: 8` that other reviews do not need. In SQL this would require altering the table. In MongoDB it just works.

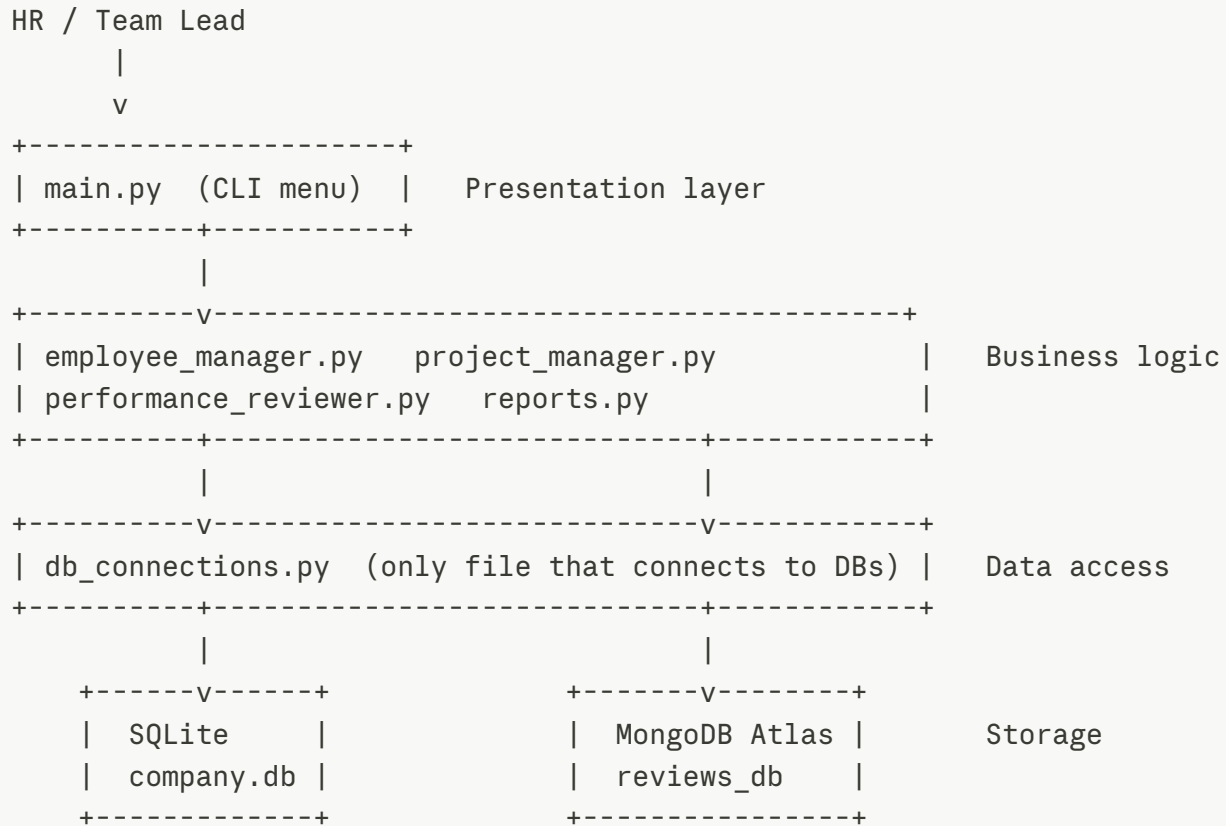
3. Tools and Technologies Used

Tool	Used for
Python 3.11.9	Application logic, validation, CLI, reports
<code>sqlite3</code> (built into Python)	Connecting to and querying the SQLite database
SQLite (<code>company.db</code>)	Stores employees, projects and assignments in one local file
MongoDB Atlas (free M0 cluster, AWS Mumbai region)	Cloud-hosted MongoDB that stores performance reviews
<code>pymongo</code>	Python driver for MongoDB (<code>insert_one</code> , <code>find</code> , <code>create_index</code>)
<code>pandas</code>	Basic data analysis: average, minimum and maximum rating
<code>python-dotenv</code>	Reads the MongoDB connection string from a <code>.env</code> file so no password is written in code
<code>pytest</code> (9.1.1)	Writing and running automated tests
<code>pytest-cov</code> (7.1.0)	Measuring test coverage
<code>mongomock</code>	A fake in-memory MongoDB so tests run without a real database
Virtual environment (<code>venv</code>)	Keeps the project libraries separate from the rest of the PC
VS Code	Code editor and built-in terminal
Git	Version control (commits, branches, push)
GitHub	Hosting the code and the README publicly

Tool	Used for
Mermaid	Diagrams (architecture, ER diagram) written as text in Markdown

4. Architecture

The application follows a layered design. Each layer only talks to the layer below it.



Supporting files: `exceptions.py` (custom errors) and `utils.py` (input validation) are used by all modules.

Why this structure? If the database changes later (for example SQLite to PostgreSQL), only `db_connections.py` and a few queries need to change. The menu and the business rules stay the same. This is called separation of concerns.

5. Database Design

5.1 SQL Design (SQLite)

Three tables are used. The PDF asks for two (Employees, Projects) and a linking table (EmployeeProjects) for assignments.

Employees

Column	Type	Constraint	Reason
employee_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique ID generated automatically
first_name, last_name	TEXT	NOT NULL	Name is mandatory
email	TEXT	NOT NULL, UNIQUE, case-insensitive	Prevents duplicate employees, even if capital letters differ
hire_date	TEXT (YYYY-MM-DD)	NOT NULL	SQLite has no DATE type, so ISO text is used; it sorts and compares correctly
department	TEXT	NOT NULL	Required information

Projects

Column	Type	Constraint	Reason
project_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique ID
project_name	TEXT	NOT NULL	Required
start_date	TEXT	NOT NULL	Required
end_date	TEXT	NULL allowed	A project may not have an end date yet
status	TEXT	DEFAULT 'Planning', CHECK in (Planning, Active, On Hold, Completed, Cancelled)	Only valid statuses allowed; new projects start as Planning

Column	Type	Constraint	Reason
		CHECK end_date >= start_date	A project cannot end before it starts

EmployeeProjects (junction table)

Column	Type	Constraint	Reason
assignment_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique ID for each assignment
employee_id	INTEGER	FOREIGN KEY to Employees, NOT NULL	Links to an existing employee
project_id	INTEGER	FOREIGN KEY to Projects, NOT NULL	Links to an existing project
role	TEXT	NOT NULL	The role on that project
assigned_date	TEXT	DEFAULT today	Date of assignment
		UNIQUE (employee_id, project_id)	Same employee cannot be assigned to the same project twice

Relationship: Employees and Projects have a **many-to-many** relationship. One employee can work on many projects, and one project can have many employees. A many-to-many relationship cannot be stored directly in two tables, so the junction table `EmployeeProjects` sits in between. Foreign keys use `ON DELETE CASCADE` so deleting an employee also removes their assignments, leaving no orphan rows.

Important detail: SQLite does not enforce foreign keys by default. The code runs `PRAGMA foreign_keys = ON` on every connection to turn enforcement on.

5.2 NoSQL Design (MongoDB)

- Database: `reviews_db` (the PDF's example name was `performance_reviews_db`; the name is controlled by the `MONGO_DB_NAME` setting in `.env`)
- Collection: `reviews`
- Each document is one performance review.

Example document from the project:

```
{
  "_id": ObjectId("6ac0af8285b6ff8da6937c0c"),
  "employee_id": 1,
  "review_date": "2026-10-03",
  "reviewer_name": "Meena S",
  "overall_rating": 4.5,
  "strengths": ["Teamwork", "Coding"],
  "areas_for_improvement": ["Documentation"],
  "comments": "Great year.",
  "goals_for_next_period": ["Lead a module"],
  "leadership_score": 8
}
```

Points worth noting:

- `strengths`, `areas_for_improvement` and `goals_for_next_period` are real lists inside the document.
- `leadership_score` is an extra field that exists only in this document. This shows schema flexibility.
- An **index on `employee_id`** is created so finding all reviews for one employee is fast. Atlas shows 2 indexes: the default `_id` index and the `employee_id` index.
- MongoDB has no foreign keys, so the code checks that the employee exists in SQLite before saving a review.

6. Code Structure: Every File Explained

File	Purpose
<code>main.py</code>	Entry point. Shows the 8-option menu, reads input with <code>input()</code> , calls other modules, and catches errors so the program never crashes
<code>db_connections.py</code>	The only file that creates database connections. Creates the three SQL tables if they do not exist (<code>CREATE TABLE IF NOT EXISTS</code>). Connects to MongoDB Atlas and returns the <code>reviews</code> collection
<code>employee_manager.py</code>	<code>add_employee</code> (handles duplicate emails), <code>get_employee_by_id</code> , <code>list_all_employees</code>

File	Purpose
<code>project_manager.py</code>	<code>add_project</code> , <code>assign_employee_to_project</code> , <code>get_projects_for_employee</code> (uses a JOIN), plus helpers <code>get_project_by_id</code> and <code>list_all_projects</code>
<code>performance_reviewer.py</code>	<code>submit_performance_review</code> (accepts any extra fields) and <code>get_performance_reviews_for_employee</code>
<code>reports.py</code>	<code>generate_employee_project_report</code> (JOIN across three tables) and <code>generate_employee_performance_summary</code> (one SQL query plus one MongoDB query, average rating with pandas)
<code>exceptions.py</code>	Custom error classes such as <code>DuplicateEmailError</code> and <code>EmployeeNotFoundError</code>
<code>utils.py</code>	Validation helpers for empty text, email format and <code>YYYY-MM-DD</code> dates
<code>seed_data.py</code>	Adds demo employees, a project, assignments and a review
<code>company.db</code>	The SQLite database file
<code>requirements.txt</code>	List of libraries to install
<code>.env.example</code>	Template showing which settings are needed (no real secrets)
<code>.env</code>	Real settings including the MongoDB connection string (never uploaded to GitHub)
<code>.gitignore</code>	Tells Git to ignore <code>venv/</code> , <code>.env</code> , cache files
<code>pytest.ini</code> , <code>.coveragerc</code>	Test and coverage configuration
<code>tests/</code>	Automated test suite
<code>docs/ARCHITECTURE.md</code>	Architecture diagram and design reasoning
<code>README.md</code>	Setup and usage instructions

7. Important Techniques and Why They Were Used

1. **Parameterized queries.** SQL uses `?` placeholders, for example `WHERE employee_id = ?`. Values are never pasted into the SQL text. This prevents **SQL injection**.
2. **with conn: transactions.** Changes are committed if everything works and rolled back if an error occurs, so the database is never left half-updated.
3. **Closing connections in finally.** Connections are always closed, even when errors occur.
4. **Handling duplicate emails.** The database `UNIQUE` constraint is the real protection. The code catches `sqlite3.IntegrityError` and raises a friendly `DuplicateEmailError`. This is safer than checking first and then inserting, because it also works if two users insert at the same time.
5. **row_factory = sqlite3.Row.** Rows can be read by column name and converted to dictionaries.
6. **JOIN queries.** Combine data from Employees, Projects and EmployeeProjects into one readable report.
7. **Custom exceptions.** Modules raise specific errors; `main.py` catches them and prints a message such as `Employee ID 99 not found.` instead of a long traceback.
8. **Environment variables with .env.** The MongoDB password stays out of the code and out of GitHub.
9. **Input validation.** Empty names, wrong email formats, impossible dates (like 2023-13-45), ratings outside 1 to 5 and invalid statuses are rejected with clear messages.
10. **Flexible reviews with **extra_fields.** The review function accepts any additional keyword arguments and stores them in the document.
11. **Test isolation.** Tests use a temporary SQLite file and `mongomock`, so real data is never touched and tests need no internet.

8. What I Did, Step by Step

1. Read the problem statement and understood the four goals and the SQL versus NoSQL requirement.
2. Checked that Python was installed and installed VS Code with the Python extension.
3. Extracted the project folder and opened it in VS Code.
4. Created a virtual environment with `python -m venv venv` and activated it with `.\venv\Scripts\activate`.

5. Installed all libraries with `pip install -r requirements.txt`.
6. Ran the test suite with `pytest --cov=. --cov-report term-missing`. Result: **40 tests passed, 96% coverage** (requirement was 80%).
7. Created a free MongoDB Atlas account and an M0 cluster named Cluster0 in the AWS Mumbai region.
8. Atlas added my IP address and created a database user automatically. I built the connection string from the username, password and cluster address.
9. Copied `.env.example` to `.env` and put the connection string in it.
10. Ran `python seed_data.py`. Result: Demo data added.
11. Ran `python main.py` and tested the menu: listing employees, viewing projects, the performance summary and the report.
12. Opened Atlas Data Explorer and confirmed the review document is stored in `reviews_db`, collection `reviews`, including the extra `leadership_score` field.
13. Created a GitHub repository, initialised Git, committed and pushed the project. Confirmed that `.env` and `venv/` were not uploaded.
14. Resolved a README conflict (see section 11) and pushed again. The final repository has 3 commits.

9. Verified Results

Performance summary for employee 1 (from MongoDB plus SQL):

```
Employee    : Asha Rao (ID 1)
Department  : Engineering
Reviews     : 1
Avg Rating  : 4.50 / 5 (min 4.5, max 4.5)
Strengths   : Coding, Teamwork
Improve     : Documentation
```

Employee-project report (SQL JOIN):

Employee Name	Project Name	Role	Assigned Date
Asha Rao	Website Revamp	Developer	2026-10-03
Ravi Kumar	Website Revamp	Tester	2026-10-03

10. Testing

- **40 automated tests** in five files: employees, projects, reviews, reports and database, and the CLI.
- **Coverage: 96%** overall. The employee, project and reports modules are at 100%. The few uncovered lines are error paths that need a real MongoDB failure.
- Tests check: successful creation, duplicate email (`pytest.raises`), retrieval of one and many records, correct assignment, duplicate assignment, missing IDs, invalid input (parametrized tests), review submission and retrieval, extra fields, report output, foreign key enforcement, and the full menu flow using scripted input.
- `conftest.py` provides shared **fixtures**. An `autouse` fixture gives every test a fresh temporary SQLite file and a fake MongoDB, which keeps tests independent.

Command: `pytest --cov=. --cov-report term-missing`

11. Problems Faced and How They Were Solved

Problem	Cause	Solution
Needed a database connection string	Atlas generates a template with placeholders <code><db_username></code> and <code><db_password></code>	Replaced both with real values and removed the angle brackets
Atlas network access	Atlas only accepts connections from approved IP addresses	Atlas added my current IP automatically; a new network needs a new entry in Network Access
Database name differed from the PDF example	The name is read from <code>MONGO_DB_NAME</code> in <code>.env</code>	Understood it is configurable; data is in <code>reviews_db</code>
Git push conflict on README.md	The GitHub repository already had a README, and my project also had one ("both added")	Used <code>git checkout --ours README.md</code> , then <code>git add</code> , <code>git commit</code> and <code>git push</code>

Problem	Cause	Solution
Keeping secrets safe	The password must not be public	Stored it in <code>.env</code> , listed <code>.env</code> in <code>.gitignore</code> , and checked <code>git status</code> before committing

12. What I Learned

Databases

- SQL is best for structured, related data. NoSQL is best for flexible, document-style data. Choosing between them depends on how the data looks and how it changes.
- How to design tables: primary keys, foreign keys, `NOT NULL`, `UNIQUE`, `DEFAULT` and `CHECK` constraints, and why each one protects data quality.
- Many-to-many relationships need a junction table.
- JOINS combine related tables in one query.
- SQLite stores dates as text, and foreign keys must be enabled with a `PRAGMA`.
- MongoDB stores JSON-like documents, each document can have different fields, and an index speeds up searches.
- MongoDB has no built-in foreign keys, so the application must check references.

Python

- Using `sqlite3` and `pymongo` to talk to databases.
- Splitting a program into modules with one responsibility each.
- Writing custom exceptions and handling errors with `try`, `except` and `finally`.
- Using `with` blocks for safe transactions.
- Using `**kwargs` to accept flexible data.
- Reading configuration from environment variables with `python-dotenv`.
- Using `pandas` for quick calculations such as mean, min and max.
- Building a menu-driven CLI with a loop and `input()`.

Security

- Parameterized queries stop SQL injection.
- Passwords belong in `.env`, never in code or on GitHub.

Testing

- Writing unit tests with pytest, using fixtures, `monkeypatch`, parametrization and `pytest.raises`.
- Testing without real services by using a temporary database and `mongomock`.
- Measuring coverage with `pytest-cov` and reading the `Missing` column.

Tools and workflow

- Creating and activating a virtual environment, and why it matters.
- Using the VS Code terminal.
- Setting up MongoDB Atlas: cluster, database user, network access and connection string.
- Git basics: `init`, `add`, `status`, `commit`, `branch`, `remote add`, `push`, and resolving a merge conflict.
- Writing a README and architecture documentation so others can run the project.

13. Possible Future Improvements

- Update and delete features for employees, projects and reviews.
- Search and filter, such as employees by department or projects by status.
- Export reports to CSV or Excel.
- A web interface using Flask or Streamlit instead of the command line.
- Login with roles (HR versus team lead).
- Charts of ratings over time.
- Pagination for large lists.
- Using an ORM such as SQLAlchemy.
- A GitHub Actions workflow to run the tests automatically on every push.

14. Quick Answers for Viva or Interview

Why SQLite for employees? The data has a fixed structure and relationships, and it needs integrity rules like a unique email and foreign keys.

Why MongoDB for reviews? Review fields vary and change over time, and documents can store lists and new fields without altering a schema.

What is the EmployeeProjects table? A junction table that turns the many-to-many relationship between employees and projects into two one-to-many relationships. It also stores the role and assigned date.

How are duplicate emails prevented? A `UNIQUE` constraint in the database. The code catches the error and shows a friendly message.

How do you avoid SQL injection? By using `?` placeholders (parameterized queries).

How do the two databases connect? Through `employee_id`. SQLite holds the employee, and MongoDB reviews store the same ID. The report makes one query to each and combines the results in Python.

How did you test without a real MongoDB? With `mongomock` and a temporary SQLite file created for each test.

Where is the password stored? In a `.env` file that is excluded from Git by `.gitignore`.

What does `PRAGMA foreign_keys = ON` do? It turns on foreign key checking, which SQLite leaves off by default.

15. How to Run the Project

```
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
(edit .env and add the MongoDB connection string)
python seed_data.py          (optional demo data)
python main.py               (start the application)
pytest --cov=. --cov-report term-missing (run tests)
```

16. Deliverables Checklist

- Working Python application (`main.py` and supporting modules): done
- SQLite database file (`company.db`): done
- Instructions and MongoDB connection details: done (README and `.env.example`)
- README.md: done
- Project documentation with architecture diagram and purpose: done (this document and `docs/ARCHITECTURE.md`)

- Automated tests with at least 80% coverage: done (96%)
- Code on GitHub: done